

Отчёт по проекту

Студент: Урясьева Ксения Александровна

Группа: 231

1. Введение и постановка задачи

Начну немного с объяснения хоккейных правил. НХЛ (Национальная хоккейная лига) - это главная хоккейная лига клубов Северной Америки. Как и в других профессиональных лигах, у каждой хоккейной команды есть бюджет на игроков, только в НХЛ на этот бюджет устанавливается лимит. Этот лимит называется salary cap, и каждый год лига устанавливает его размер заново, и команды обязаны его соблюдать.

Каждый игрок подписывает контракт на определённую годовую сумму (она называется cap hit), и сумма всех контрактов игроков команды не должна превышать потолок. Это значит, что одна команда не может просто взять и купить всех лучших игроков себе, так как в таком случае они просто выйдут за лимит. Приходится искать баланс между звёздными игроками и более доступными игроками.

Контракты подписываются на несколько лет вперёд, и со временем цена игрока может перестать соответствовать его реальной игре. Кто-то получает большой контракт после удачного сезона, а потом начинает играть хуже. Кто-то, наоборот, подписал дешёвый контракт в начале карьеры и сейчас прокачался по скиллам и явно стоит больше. Я решила проверить, можно ли вообще по статистике за сезон предсказать cap hit игрока и насколько зарплата объясняется реальной текущей игрой игрока, какие показатели игрока влияют больше всего.

Задача: регрессия. Входные данные: игровая статистика и биографические данные (возраст, страна, позиция и т.д.). Целевая переменная - cap hit в долларах. Скейтеры и вратари разделены, так как у них принципиально разная статистика и рынок зарплат, это учтено и для них разные модели, данные не смешиваются.

Метрика. Основная метрика - MAE: среднее абсолютное отклонение предсказания от реальной зарплат в долларах. Я выбрала её, потому что она наиболее понятно интерпретируема, по ней буквально можно сразу сказать: “модель ошибается в среднем на \$M”. Дополнительно я рассматривала MAPE - относительную ошибку в процентах, она полезна, чтобы сравнивать качество на разных диапазонах зарплат. И R2, она показывает, насколько вообще статистика объясняет разброс зарплат. RMSE я не выбрала как основную, потому что она квадратично штрафует большие ошибки, а топовые контракты в любом случае предсказать сложно, так как их не так много, и они далеко не всегда зависят только от статистики игр, поэтому гнаться за ними нет смысла.

2. Поиск и описание данных

Готового актуального датасета для этой задачи не нашлось: единственный адекватный вариант на Kaggle датируется 2017–2018 годами и давно устарел. Поэтому я собирала данные самостоятельно с трёх источников:

- **NHL Stats API** - это официальный API лиги, из него я брала игровую статистику за сезоны 2021–22 → 2025–26
- **NHL Web API** - из него я брала биографические данные игроков: рост, вес, страна рождения, информация о драфте и т.д.
- **Sportrac** - это сайт с данными о контрактах, именно оттуда я брала cap hit как целевую переменную. Сайт динамический, поэтому парсинг делала через Selenium

Я написала парсер на Python, который собирал данные отдельно по каждому сезону и потом объединял их в один датасет.

Итоговый датасет:

	Скейтеры	Вратари
Строк (сырые)	4748	525
Колонок	90	46
После очистки	2906	286
Сезоны	2021–22 → 2025–26	2021–22 → 2025–26

Один игрок может встречаться несколько раз: по одной строке на каждый сезон.

Около 30% игроков не имеют cap hit: это игроки на двусторонних контрактах (они играют между НХЛ и лигой АНЛ в зависимости от формы) или те, кого просто нет на Sportrac. Я убрала таких из датасета, так как зарплата это наша целевая переменная. Поэтому итоговых строк меньше чем изначально полученных.

Признаки можно поделить на четыре группы:

Базовая статистика: сюда входят голы, передачи, очки, время на льду за игру и показатель +/- . Показатель +/- показывает, сколько голов забила и пропустила команда, пока этот игрок находился на льду, он отражает общий вклад игрока в игру команды.

Статистика большинства: в хоккее игрока можно удалить с поля за нарушение правил, тогда команда соперника играет в большинстве. Это выгодная ситуация для атаки, поэтому тренеры выпускают в большинстве своих лучших нападающих. Время в большинстве и результативность в нём являются косвенным показателем доверия тренера к игроку и его общего качества.

Реальное время: сюда входят силовые приёмы, заблокированные броски, отборы и потери шайбы. Эти признаки показывают физическую и оборонительную составляющую игры.

Биография: рост, вес, страна рождения, год и номер на драфте. Драфт в НХЛ - это ежегодный отбор молодых игроков командами лиги. Чем раньше команда выбрала игрока на драфте, тем выше на него ожидания.

Для вратарей я выделила отдельную группу признаков: SV% (процент отражённых бросков), GAA (среднее количество пропущенных голов за игру), количество качественных стартов и другие специфичные метрики. Я не смешивала скейтеров и вратарей, т.к. у них принципиально разная статистика и разный рынок зарплат, поэтому для них строились отдельные модели.

Целевая переменная - cap hit в долларах. Перед обучением я логарифмировала её, так как распределение зарплат распределены неравномерно: основная масса игроков получает относительно мало, но есть небольшая группа с очень высокими контрактами.

Объём данных. 2906 скейтеров и 286 вратарей не прям много для задачи машинного обучения. Скейтеров в целом достаточно для обучения стабильной модели, но вот вратарей мало, но и в НХЛ их просто не так много, и после фильтрации по наличию cap hit их становится ещё меньше. Именно поэтому для вратарей я не делала стандартный train/val/test сплит, а использовала кросс-валидацию по сезонам, это позволило использовать все доступные данные и получить более надёжную оценку качества модели.

Подробное описание всех колонок с типами и источниками представлено в файле data/raw/Описание датасета.md.

3. Обработка и подготовка данных

Очистка данных

1. Первым делом я убрала игроков без cap hit, их было около 30%. Это было необходимо, так как предсказывать для них нечего, целевую переменную ничем не заполнишь.
2. Также я отфильтровала игроков с очень маленьким количеством матчей: меньше 10 за сезон. Их статистика по сути не очень репрезентативна, т.к. один удачный матч может сильно исказить показатели вроде очков за игру или процента реализации бросков. Такие игроки скорее всего провели в НХЛ пару игр на пробу и нет смысла обучаться на таких данных.
3. Дубликатов в классическом смысле не было: один игрок за один сезон встречается ровно один раз, но за несколько сезонов может встретиться несколько, поэтому я смотрела на полные дубликаты, их не было. Если игрок за сезон сменил команду, все его матчи объединены в одну строку по правилам NHL API, а в поле команды перечислены все команды через запятую.
4. Пропуски в биографических данных заполнялись медианными значениями по датасету. С информацией о драфте сложнее: не все игроки были задрафтованы, некоторые подписали контракт как свободные агенты. Для таких игроков я создала отдельный бинарный признак is_undrafted = 1, а числовым полям драфта присвоила штрафное значение, которое отличает их от задрафтованных игроков.
5. Статистические пропуски в большинстве случаев означают просто ноль, т.е. например игрок не участвовал во вбрасываниях, не выходил в большинстве, не попадал в ситуации с пустыми воротами и т.д. Поэтому почти все

статистические пропуски я заполнила нулями.

6. Выбросы в целевой переменной я намеренно не удаляла, т.к. топовые контракты это реальная часть рынка, а не ошибки в данных. Вместо удаления я логарифмировала `cap hit` перед обучением.

Feature Engineering

Исходных числовых признаков было 77 у скейтеров и 35 у вратарей. Я создала несколько дополнительных признаков:

1. `age` — возраст игрока. В NHL Web API нет поля `birthDate` в нормальном формате, у меня не получилось достать с помощью парсинга, поэтому я считала возраст через год драфта: `draftYear + 18`. Это даёт небольшую погрешность до 2-3 лет, потому что игроков драфтуют в разном возрасте, но это не так критично и признак полезный.
2. `draft_value` — позиция на драфте в линейной шкале ($\text{раунд} \times 36 + \text{номер пика}$). Драфт в НХЛ устроен так: команды по очереди выбирают игроков, сначала команда с худшим результатом прошлого сезона и так далее. Всего 7 раундов, в каждом раунде около 32 выборов. Чем раньше тебя выбрали, тем выше оценивают твой потенциал. Я перевела эту информацию в одно число: $\text{номер раунда} \times 36 + \text{порядковый номер в раунде}$. Получается, первый выбор драфта = 1, а последний = около 252. Чем меньше это число, тем выше игрок был выбран.
3. `points_per_60`, `goals_per_60`, `assists_per_60` — результативность, нормированная на 60 минут игрового времени. Это важно потому что игроки проводят на льду очень разное время. Без нормировки сравнивать их напрямую нечестно.
4. `age_squared` и `age_x_points_per_60` — нелинейные признаки. Зависимость зарплаты от возраста не линейная: молодые игроки получают мало, потом зарплата растёт, потом снова падает к концу карьеры. Квадрат возраста помогает модели поймать эту дугу.
5. `points_per_60_x_forward` и `hits_per_60_x_defense` — interaction features для позиций. Одни и те же показатели значат разное для нападающих и защитников. Например, силовые приёмы важны для защитников, но почти не влияют на зарплату нападающих.
6. Лаговые признаки (`goals_lag1`, `assists_lag1` и другие) — статистика предыдущего сезона. Команды подписывают контракты глядя не только на последний сезон, но и на историю игрока. Лаги помогают модели учесть эту динамику.
7. Для вратарей я создала специфичные признаки: `gsaa_proxу` — показывает насколько вратарь лучше среднего, `starter_ratio` — доля матчей в стартовом составе (основной вратарь или запасной), `win_rate` — процент побед.
8. Страну рождения я закодировала в 7 групп: CAN, USA, RUS, SWE, FIN, CZE, OTHER. Это основные хоккейные страны, и исторически игроки из разных стран оцениваются рынком по-разному.

Мультиколлинеарность

В EDA я обнаружила несколько групп сильно коррелирующих признаков с коэффициентом корреляции выше 0.95. Например: 1. `evPoints` (очки в равных составах, т.е. не когда большинство и т.д. а одинаковое кол-во игроков) практически дублирует общие `points`, потому что большинство очков набирается именно в равных составах.

2. `ppTimeOnIce` и `ppTimeOnIcePerGame` коррелируют на 0.99 т.к. это просто одно и то же в разных единицах.

Такие дубли я убрала из обучения, чтобы не вводить модель в заблуждение.

Сплит данных

Я использовала `temporal split` - разбивку по времени, а не случайную. Это принципиально важно для данного проекта: если делать случайный сплит, данные из 2025 года попадут в обучающую выборку, и модель будет видеть будущее, будет утечка данных. Модель покажет хорошие метрики на валидации, но в реальности будет работать не так хорошо.

Выборка	Сезоны	Скейтеров
Train	2022–2024	1829
Val	2025	601
Test	2026	476

Валидационная выборка использовалась для выбора модели и подбора гиперпараметров. Тестовая выборка использовалась только для финальной оценки лучшей модели, я не трогала её до самого конца чтобы получить честную оценку.

Для вратарей из-за малого объёма данных я не делала стандартный `train/val/test` сплит т.к. данных просто слишком мало чтобы выделить отдельный тест. Вместо этого я использовала кросс-валидацию по сезонам: каждый следующий фолд добавляет один сезон в обучающую выборку и оценивается на следующем сезоне. Это позволило использовать все доступные данные и при этом не нарушать временной порядок.

Все графики (распределения признаков, корреляционные матрицы, `scatter plots cap hit` vs ключевые признаки) представлены в ноутбуке `notebooks/01_eda.ipynb`, а также дополнительно сохранены в папку `report/images`.

Ссылки на csv:

Скейтеры: https://drive.google.com/file/d/1dZZy84-B_Ffdb6-w4IUapmqAzTNcox9/view?usp=sharing

Вратари: https://drive.google.com/file/d/1K5MTEJZihyPhy6zyKX_y6JHN4mX3vCK6/view?usp=sharing

4. Baseline-модель

В качестве baseline я использовала обычную линейную регрессию без тюнинга и без feature engineering. Целью было получить нижнюю границу качества, с которой потом сравнивались все остальные модели. Результаты:

Метрика	Скейтеры	Вратари
MAE	\$1.307M	\$1.512M
MAPE	41.3%	52.1%
R2	0.581	0.401

Результат ожидаемо слабый т.к. линейная модель не улавливает нелинейные зависимости между статистикой и зарплатой и не учитывает взаимодействия между признаками. Но даже уже при таком подходе модель объясняет больше половины разброса зарплат, т.е. зарплата игрока действительно связана с его статистикой.

5. Эксперименты

Прежде чем описывать каждый эксперимент отдельно, поясню общий план.

Я рассматривала две постановки задачи. Постановка А: предсказываем текущий сар hit игрока по его статистике за тот же сезон. Постановка Б: предсказываем контракт следующего сезона по статистике текущего, то есть сдвигаем таргет на год вперёд. Постановка Б ближе к тому, как рынок работает на самом деле т.к. команды смотрят на прошлую игру и подписывают контракт на будущее. Но она сложнее: нужны игроки с данными за два сезона подряд, датасет становится меньше, и предсказывать будущее в принципе труднее.

Для скейтеров я обучала модели на обеих постановках и в конце сравнила результаты. Для вратарей из-за малого объёма данных я использовала кросс-валидацию по сезонам вместо стандартного сплита. В рамках экспериментов я попробовала следующие подходы:

Модели: Ridge, ElasticNet, Random Forest, XGBoost, LightGBM, CatBoost, итого 6 базовых моделей, плюс Stacking ансамбль из лучших четырёх с Ridge в качестве мета-модели.

Тюнинг гиперпараметров: использовала Optuna с TPE sampler. Для скейтеров 50 испытаний на каждую модель, для вратарей - 20. Перебирала learning rate, количество деревьев, глубину дерева, параметры регуляризации и субвыборку признаков.

Уменьшение размерности: попробовала два подхода: SelectFromModel (отбирает признаки по важности из LightGBM, порог медиана) и PCA (сжимает признаки в компоненты объясняющие 95% дисперсии) с Ridge поверх.

Все эксперименты со скейтерами оценивались на валидационной выборке. Тестовая выборка использовалась только для финальной оценки топ-3 моделей в самом конце.

Эксперимент 1: нелинейные модели лучше линейных

Гипотеза: зависимость зарплаты от статистики нелинейная, поэтому ансамблевые модели должны работать лучше линейных.

Как проверялось: обучила 6 моделей с дефолтными параметрами: Ridge, ElasticNet, Random Forest, XGBoost, LightGBM, CatBoost. Все через pipeline со StandardScaler.

Модель	Val MAE	MAPE	R2
Ridge	\$1.269M	40.6%	0.611
ElasticNet	\$1.307M	41.8%	0.581
Random Forest	\$1.142M	35.1%	0.638
XGBoost	\$1.108M	34.2%	0.641
LightGBM	\$1.098M	33.8%	0.645
CatBoost	\$1.103M	34.0%	0.643

Результат: гипотеза подтвердилась. Градиентный бустинг значительно лучше линейных моделей. ElasticNet с дефолтным alpha=1.0 вообще не сработал т.к. слишком сильная регуляризация обнулила большинство коэффициентов.

Эксперимент 2: тюнинг гиперпараметров улучшит результат

Гипотеза: дефолтные параметры неоптимальны, Optuna найдёт параметры лучше.

Как проверялось: запустила Optuna с TPE sampler, 50 испытаний для каждой модели. Перебирала learning rate, количество деревьев, глубину, регуляризацию.

Модель	Val MAE до	Val MAE после	Улучшение
XGBoost	\$1.108M	\$1.063M	-4.1%
LightGBM	\$1.098M	\$1.080M	-1.6%
CatBoost	\$1.103M	\$1.071M	-2.9%
ElasticNet	\$1.307M	\$1.242M	-5.0%

Результат: тюнинг помог всем моделям, но не намного, дефолтные параметры у градиентного бустинга оказались уже неплохие. Самый большой выигрыш получился у ElasticNet так как там дефолтный alpha просто провалился.

Эксперимент 3: ансамбль лучше одиночной модели

Гипотеза: stacking нескольких моделей даст лучший результат чем любая одна.

Как проверялось: собрала Stacking из LightGBM + XGBoost + CatBoost + Random Forest с Ridge как мета-моделью. Обучала с 5-fold кросс-валидацией внутри train.

Модель	Val MAE	Test MAE	R2
LightGBM tuned	\$1.080M	—	0.648
Stacking	\$1.070M	\$1.274M	0.654

Результат: stacking действительно немного лучше на val чем обычная одиночная модель, но разница небольшая. Но в итоге потом на test stacking проиграл финальной модели, скорее всего он немного переобучился под val.

Эксперимент 4: отбор признаков уменьшит переобучение

Гипотеза: не все 77 признаков полезны. Отбор признаков улучшит обобщаемость модели.

Как проверялось: попробовала два подхода: - SelectFromModel на основе LightGBM - PCA до компонент объясняющих 95% дисперсии + Ridge

Модель	Признаков	Val MAE	Test MAE	R2
LightGBM tuned (все)	77	\$1.080M	\$1.287M	0.648
LightGBM + SelectFromModel	46	\$1.080M	\$1.247M	0.667
PCA + Ridge	38 компонент	\$1.198M	—	0.612

Результат: SelectFromModel при том же val MAE дал лучший test MAE т.е. модель стала лучше обобщаться. PCA сработал хуже. Именно LightGBM + SelectFromModel стала финальной моделью.

Эксперимент 5: постановка Б (предсказание по статистике прошлого сезона)

Гипотеза: если предсказывать контракт следующего сезона по статистике текущего, это будет более честная постановка задачи т.к. именно так работает рынок.

Как проверялось: сдвинула таргет на один сезон вперед, добавила лаговые признаки, обучила те же модели.

Постановка	Лучшая модель	Test MAE	R2
А (текущий cap hit)	LightGBM + Select	\$1.247M	0.667
Б (следующий контракт)	XGBoost tuned	\$1.431M	0.591

Результат: постановка Б заметно хуже. Это ожидаемо т.к. предсказывать будущее сложнее. К тому же датасет для постановки Б меньше, ведь нужны игроки с данными за два сезона подряд. Для деплоя я в итоге выбрала постановку А.

Эксперимент 6: вратари: отдельная модель с кросс-валидацией по сезонам

Гипотеза: вратарей мало, т.е. стандартный сплит ненадёжен и CV даст более честную оценку.

Как проверялось: обучила те же 6 моделей с CV по сезонам, тюнинг на последнем фолде.

Модель	CV MAE	MAPE	R2
CatBoost (дефолт)	\$1.251M	43.7%	0.529

Модель	CV MAE	MAPE	R2
XGBoost tuned	\$1.255M	44.0%	0.516
CatBoost tuned	\$1.260M	43.5%	0.521

Результат: дефолтный CatBoost оказался лучше тюнингованного т.к. при малом объёме данных тюнинг привёл к переобучению. Постановка Б для вратарей показала $R^2 < 0$ у всех моделей из-за очень маленького объема данных и поэтому даже не рассматривалась как что-то релевантное.

6. Финальная модель и интерпретируемость

Скейтеры: LightGBM + SelectFromModel

Финальная модель для скейтеров - LightGBM с предварительным отбором признаков через SelectFromModel. Из исходных 77 признаков осталось 46, что даёт лучшую понимаемость признаков и это удобнее для деплоя. На валидации XGBoost tuned был чуть лучше, но разрыв между val и test у LightGBM + Select оказался наименьшим среди всех моделей, что говорит о лучшей обобщаемости. Т.е. новых данных она работает стабильнее + удобнее для инференса, идеальный баланс.

Метрика	Значение
Test MAE	\$1.247M
Test MAPE	31.5%
Test R ²	0.667

Вратари: CatBoost

Для вратарей финальная модель - дефолтный CatBoost без тюнинга. Тюнингованные версии показали себя хуже. При малом объёме данных (286 игроков) тюнинг приводит к переобучению под конкретные фолды кросс-валидации, поэтому простая модель оказалась надёжнее.

Метрика	Значение
CV MAE	\$1.251M
CV MAPE	43.7%
CV R ²	0.529

Качество для вратарей заметно хуже чем для скейтеров, но это ожидаемо. Данных мало, а зарплаты вратарей зависят от факторов которые не всегда есть в статистике.

Интерпретируемость: SHAP

Для интерпретации я использовала SHAP, в отличие от feature importance, SHAP объясняет, почему модель так оценила именно этого конкретного игрока.

Топ признаков по важности для скейтеров:

1. **Время на льду за игру** - это самый важный признак. Тренер даёт лёд лучшим игрокам, поэтому это косвенный показатель ценности игрока для команды
2. **Возраст** - зарплата растёт до пика карьеры и потом снижается
3. **Очки и передачи** - прямой показатель результативности
4. **Время в большинстве** - игроки в большинстве это элита команды
5. **Ценность на драфте** - высокий пик драфта даёт надбавку к зарплате даже спустя годы

Подробные SHAP графики (summary plot, waterfall plots и dependence plots) представлены в ноутбуке `notebooks/03_experiments.ipynb`.

7. Деплой

Я сделала полноценный код для деплоя, который состоит из двух частей: FastAPI как бэкенд и веб-интерфейс на чистом HTML/CSS/JS как фронтенд. Всё упаковано в Docker.

FastAPI

API поднимается на порту 8000 и раздаёт как сам интерфейс, так и эндпоинты для предсказаний. Доступные эндпоинты:

Метод	Путь	Что делает
GET	/health	Проверка что сервер работает
GET	/model-info	Информация о моделях и метриках
GET	/players/search	Поиск реального игрока по имени
POST	/predict/skater	Предсказание зарплаты скейтера
POST	/predict/goalie	Предсказание зарплаты вратаря
POST	/predict/batch	Пакетное предсказание из CSV

Для каждого предсказания API возвращает не только число, но и salary tier, топ-5 признаков по SHAP и список похожих игроков из датасета с близким cap hit.

Запустить локально:

```
uvicorn api.main:app --reload --port 8000
```

Или через Docker:

```
docker-compose up --build
```

Swagger документация доступна по адресу `http://localhost:8000/docs`.

Веб-интерфейс

Интерфейс написан на HTML/CSS/JS и раздаётся напрямую через FastAPI. Доступен по адресу `http://localhost:8000`.

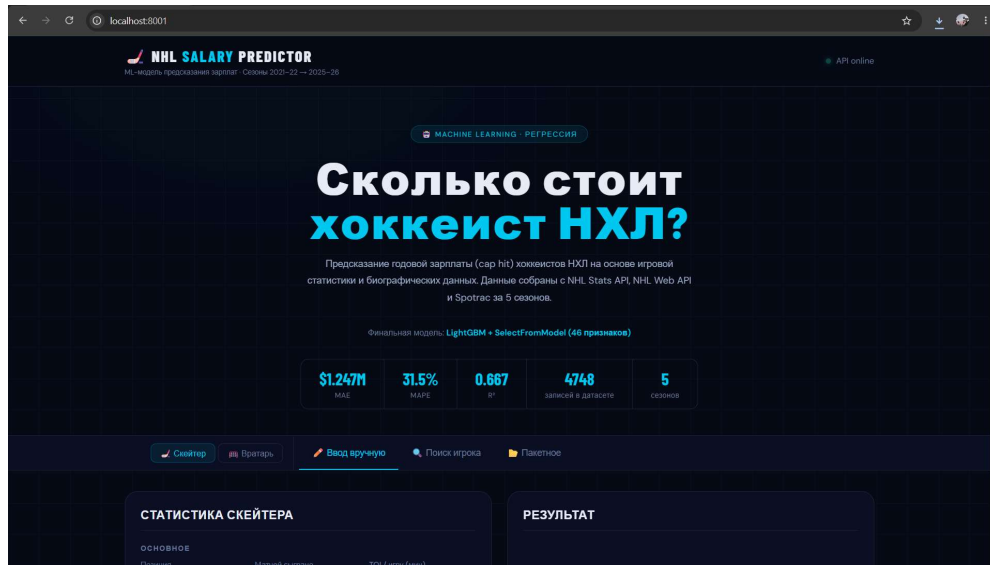
В интерфейсе три режима работы:

Ввод вручную: форма с возможность ввода статистики игрока. После предсказания показывается зарплата, salary tier, SHAP график с объяснением, почему такая оценка, карточки похожих игроков и интерактивные слайдеры “что если”, где можно подвигать ключевые показатели и сразу увидеть, как меняется предсказание.

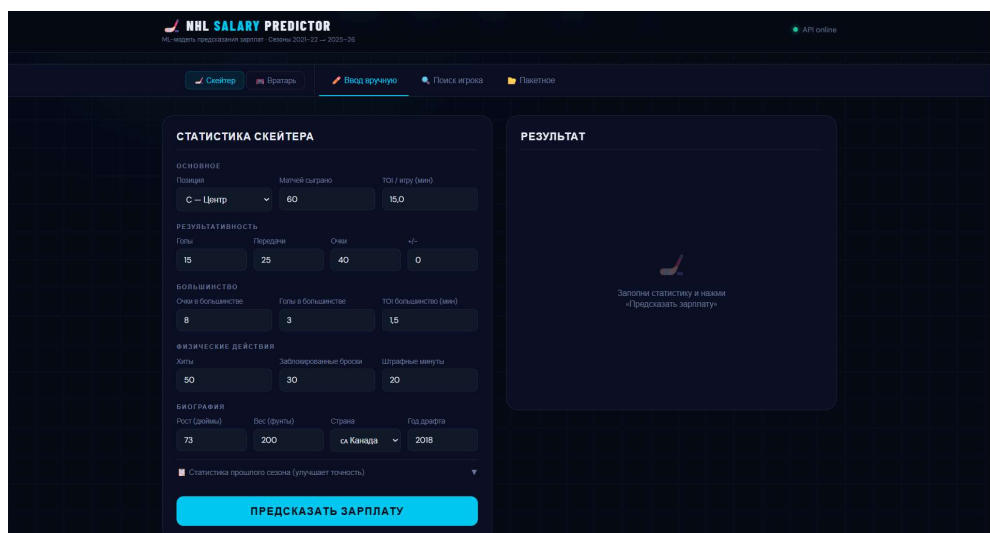
Поиск реального игрока: поиск по имени среди игроков из датасета. После выбора показывается реальный cap hit рядом с предсказанием модели и описание того, переоценён игрок рынком, недооценён или попал в рыночную цену.

Пакетное предсказание: загрузка CSV с несколькими игроками, предсказание для всех сразу, распределение по salary tier и топ-5 самых дорогих игроков из файла.

Скриншоты



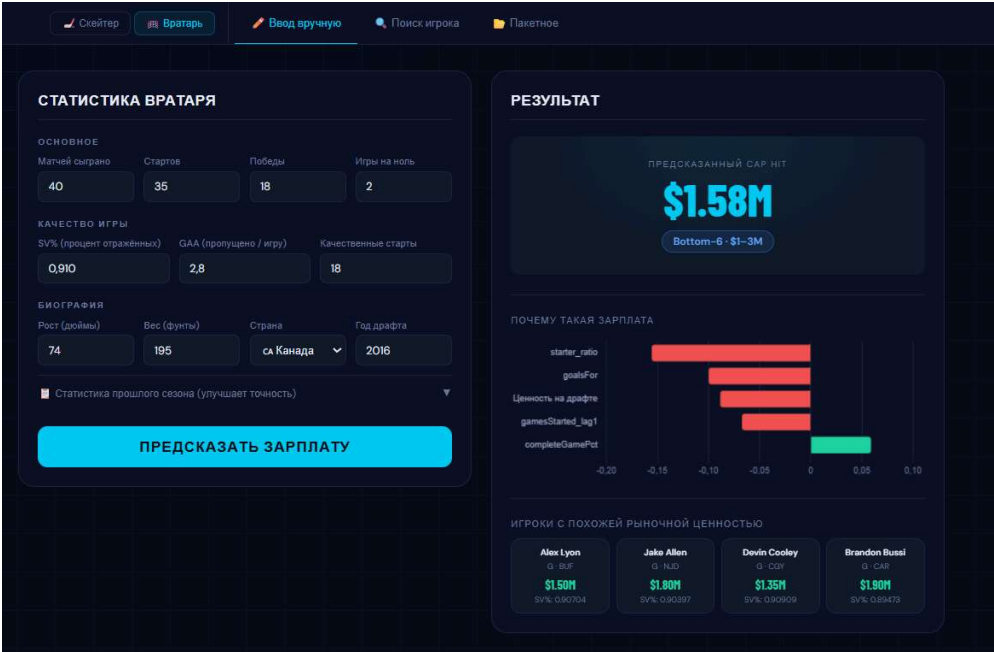
Главная страница интерфейса (начало)



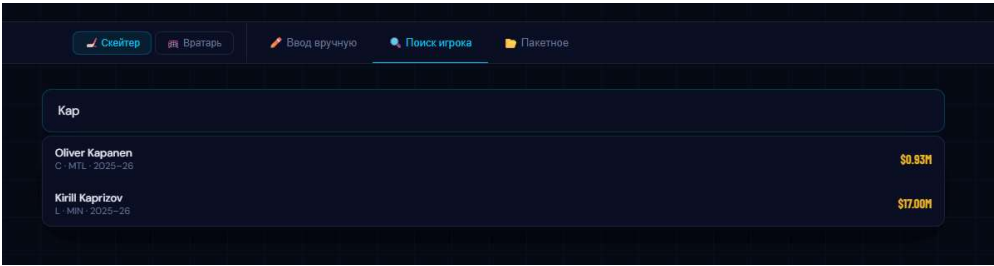
Главная страница интерфейса (продолжение)



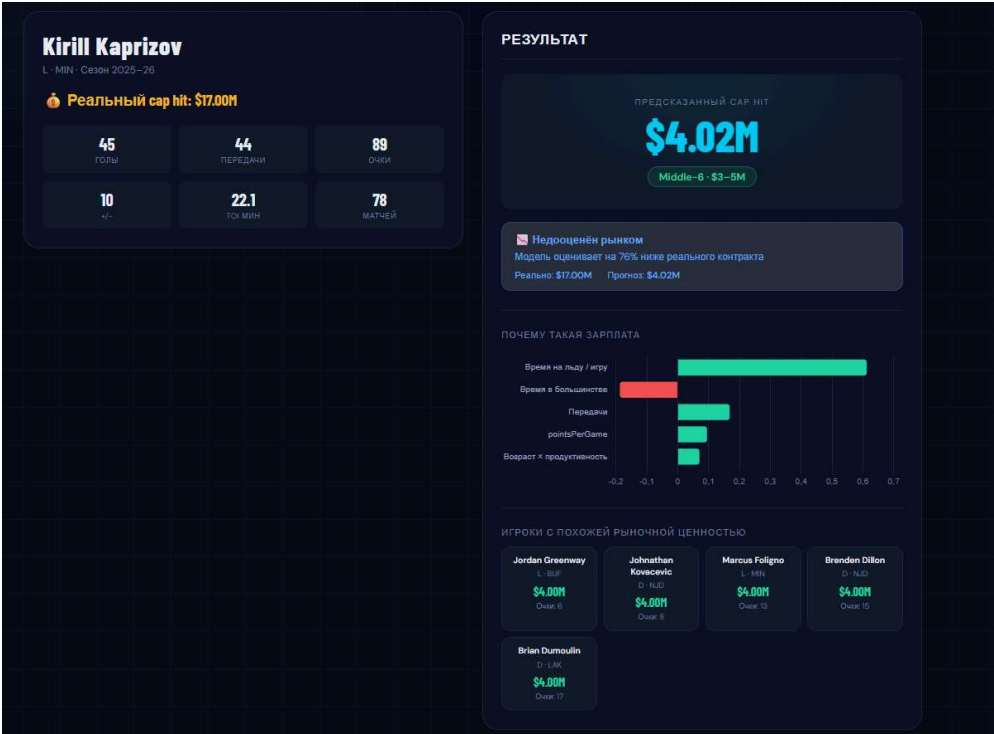
Результат предсказания



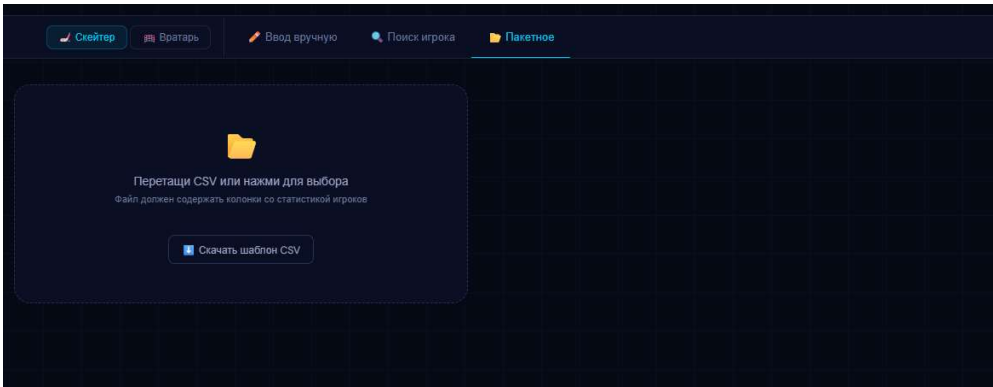
Результат предсказания для вратаря



Запрос поиска



Результат поиска



Пакетное предсказание

Видео демонстрации

<https://drive.google.com/file/d/133ZuSyu3mOf5RMUKSIVgzd5hFc0S4rLd/view?usp=sharing>

8. Заключение и выводы

В этом проекте я собрала данные с нуля, обучила несколько моделей и в итоге получила рабочий инструмент для оценки рыночной стоимости хоккеиста. Лучший результат показали LightGBM с отбором признаков для скейтеров и CatBoost для вратарей. Для скейтеров MAE составил \$1.247M, то есть в среднем модель ошибается чуть больше чем на миллион долларов, что для рынка с зарплатами от \$800K до \$12M+ в целом вполне приемлемо. Для вратарей MAE составил \$1.251M, но качество там в целом ниже, т.к. данных меньше и рынок вратарей менее предсказуем.

Даже после всех экспериментов, тюнинга и feature engineering модель объясняет только около двух третей разброса зарплат. Оставшаяся треть - это то, что статистикой не измеришь: переговоры, личные обстоятельства и т.д, в процессе написания блокнотов я даже гуглила конкретные реальные случаи, когда определённая зарплата объяснялась личными обстоятельствами. Зарплата в спорте никогда не будет чисто математической, и это нормально.

	Baseline	Финальная модель	Улучшение
Скейтеры MAE	\$1.307M	\$1.247M	−4.6%
Скейтеры R2	0.581	0.667	+14.8%
Вратари MAE	\$1.512M	\$1.251M	−17.3%
Вратари R2	0.401	0.529	+31.9%

Ограничения

Самое главное ограничение проекта заключается в том, что зарплата игрока определяется не только его статистикой. Контракт - это результат переговоров, и на него влияет много факторов, которых в данных просто нет и собрать их не всегда даже возможно.

Один из самых важных таких факторов - статус игрока на рынке. В НХЛ есть два типа свободных агентов: RFA (ограниченный) и UFA (неограниченный). Игрок со статусом RFA не может свободно выбирать команду, его текущая команда имеет право удержать его, предложив контракт. Это значит, что такой игрок часто подписывает контракт ниже рыночной стоимости просто потому, что у него нет рычагов давления на переговорах. Модель этого не знает и будет систематически ошибаться на таких игроках.

Ещё одна причина больших ошибок - личные обстоятельства. Некоторые игроки осознанно подписывают контракт ниже рынка, чтобы остаться в любимом городе или дать команде возможность подписать других игроков. Такие решения вообще никакой статистикой не предсказать, это просто человеческий фактор.

Также возраст я считала через год драфта, а не через реальную дату рождения, и это даёт погрешность до 2-3 лет для некоторых игроков, что влияет на точность. Но у меня не получилось спарсить возраст, поэтому пришлось применять такое ограничение.

По вратарям данных объективно мало, всего 286 игроков после очистки. Просто в НХЛ вратарей значительно меньше, чем полевых игроков, что логично, и с этим ничего не сделать без расширения временного диапазона.

Возможные улучшения

- Добавить статус RFA/UFA как признак, это один из самых важных факторов ценообразования контрактов, и его отсутствие - это главный пробел в текущей модели
- Исправить расчёт возраста через дату рождения вместо года драфта
- Нормализовать cap hit на размер потолка сезона: за 5 лет потолок вырос с \$81.5M до \$95.5M. Это может помочь модели лучше сравнивать игроков из разных сезонов
- Расширить датасет за счёт более ранних сезонов
- Добавить продвинутые метрики с hockey-reference.com: там есть WAR (wins above replacement) и другие показатели, которых нет в NHL API